

Healthcare Clinical Intelligence System

A 5-agent clinical pipeline behind three role-based dashboards — Patient, Doctor, and Radiologist/Lab — built on a FastAPI backend and a React single-page frontend, with SQLite as the single source of relational truth and ChromaDB holding only embeddings for retrieval.

FastAPI

SQLite

ChromaDB

OpenAI API

React + Vite

TanStack Query

Tailwind

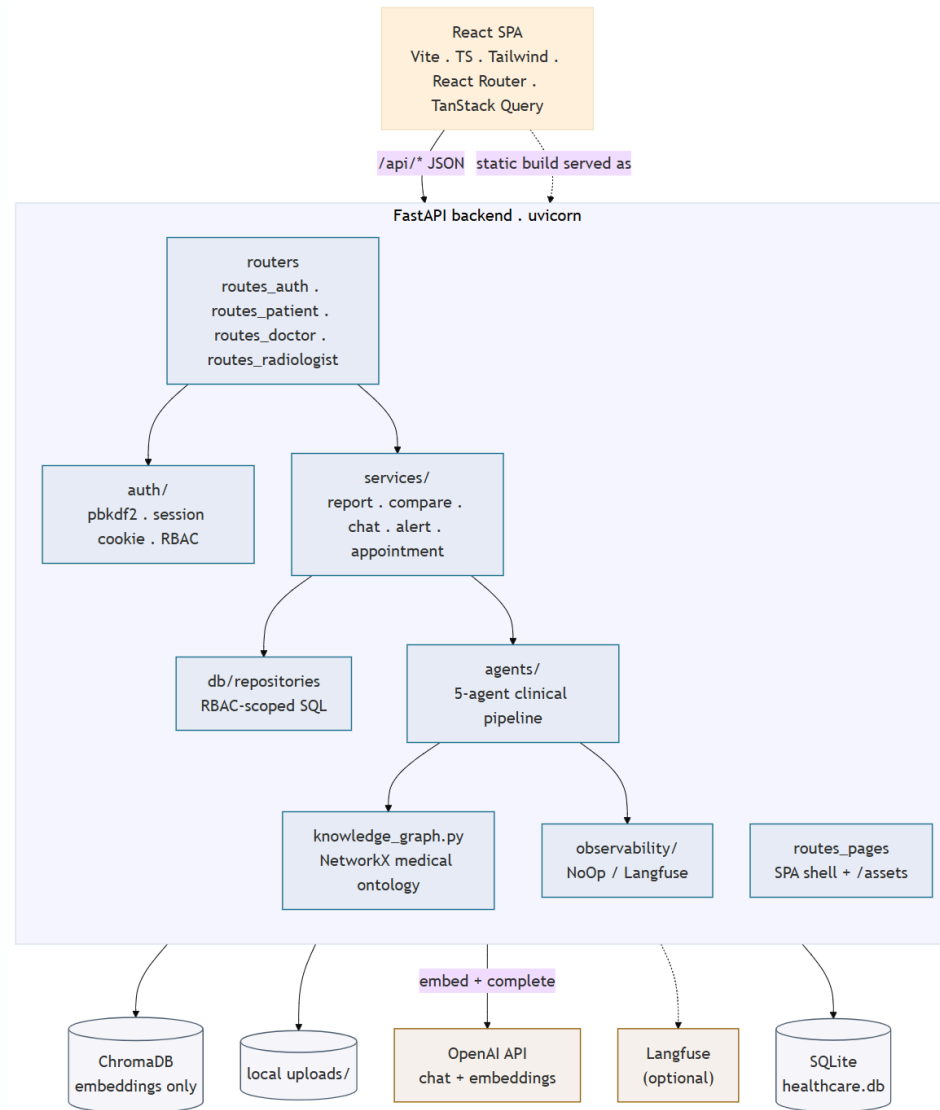
Langfuse (optional)

CONTENTS

01	System components	4.4	Doctor review → patient reveal
02	The 5-agent clinical pipeline	4.5	Patient books an appointment
03	Roles & shared data	4.6	Patient cancels an appointment
4.1	Login	4.7	AI assistant chat (RAG)
4.2	Report upload & the pipeline	4.8	Doctor ↔ radiologist messaging
4.3	Critical finding → escalation		

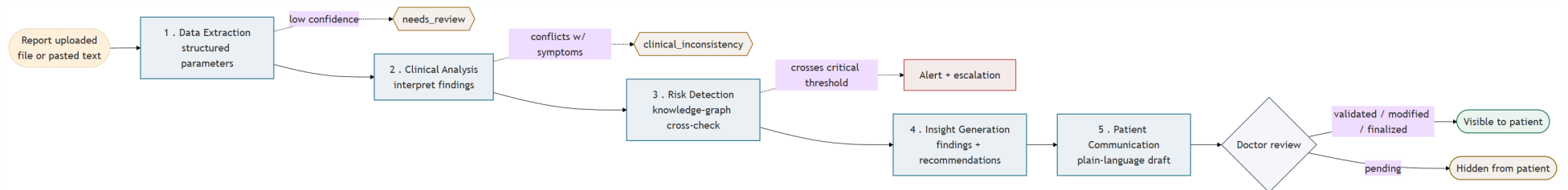
01 System components

The React SPA talks to the backend purely through `/api/*` JSON over a signed session cookie — it never touches SQLite, the vector store, or the LLM directly. FastAPI's routers stay thin and delegate to a services layer, which is the only code that reaches into repositories, the agent pipeline, and the vector store.



02 The 5-agent clinical pipeline

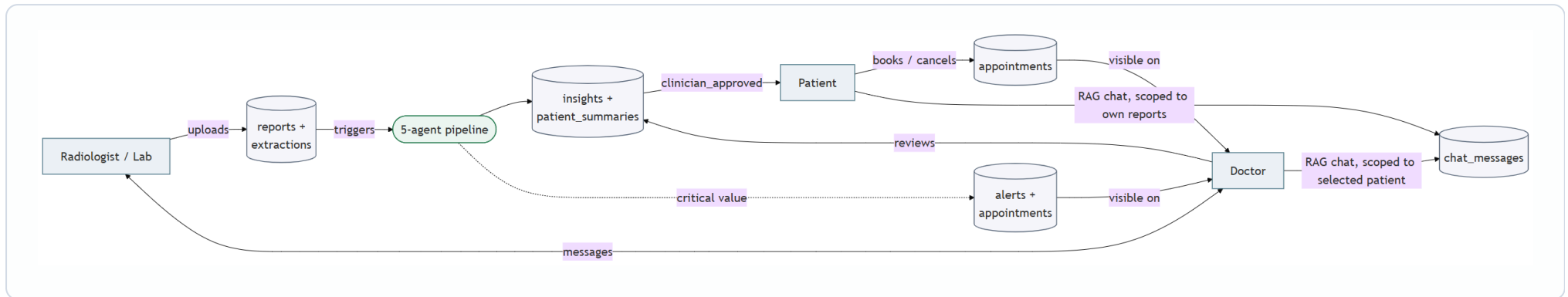
Every uploaded report runs through a fixed chain of five agents. Each stage has a deterministic negative-scenario check running alongside the LLM call, so a low-confidence extraction or an inconsistent finding is caught in code rather than trusted from the model's own self-report — and a plain-language summary never reaches the patient until a doctor has explicitly reviewed it.



□ Pipeline agent (LLM call + deterministic check) □ Negative-scenario flag / held-back state □ Critical escalation □ Reaches the patient

03 Roles & shared data

The three dashboards are separate SPA routes over the same API, scoped by session role. They don't talk to each other directly — they meet through the records each one writes: a radiologist's upload becomes a doctor's review, which becomes a patient's summary.



● Radiologist / Lab

Uploads reports

`/api/radiologist/reports/upload` , corrects extraction text, messages doctors.

● Doctor

Reviews insights

`/api/doctor/insights/:id/review` , acknowledges alerts, holds the only gate before a summary reaches a patient.

● Patient

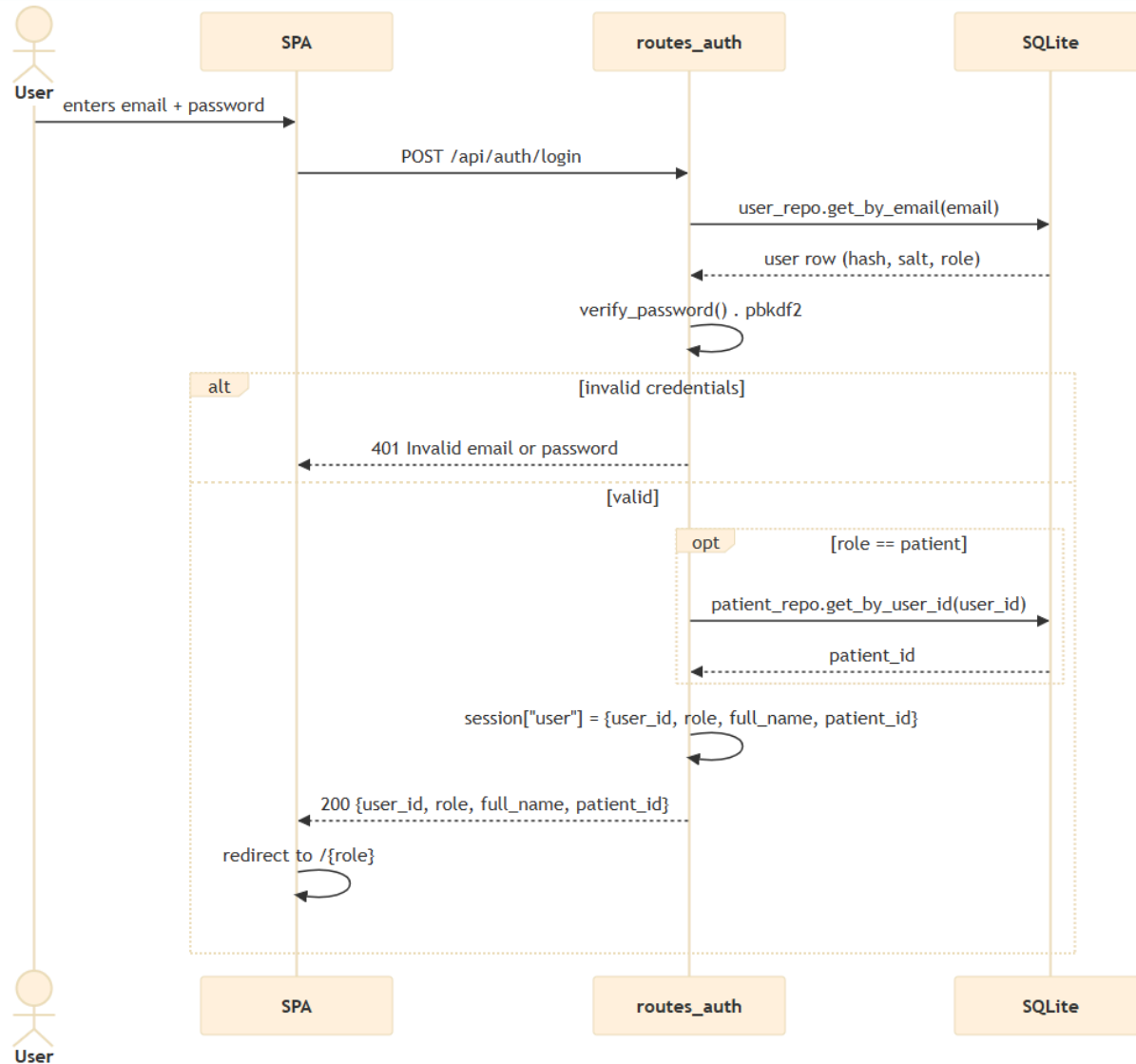
Views reports once approved, books/cancels appointments in fixed 15-minute slots, asks the RAG assistant about their own results only.

04 **Sequence diagrams**

Sections 01–03 show what the system is made of. These show what happens, in order, for each of the flows a user actually triggers — request by request, table by table.

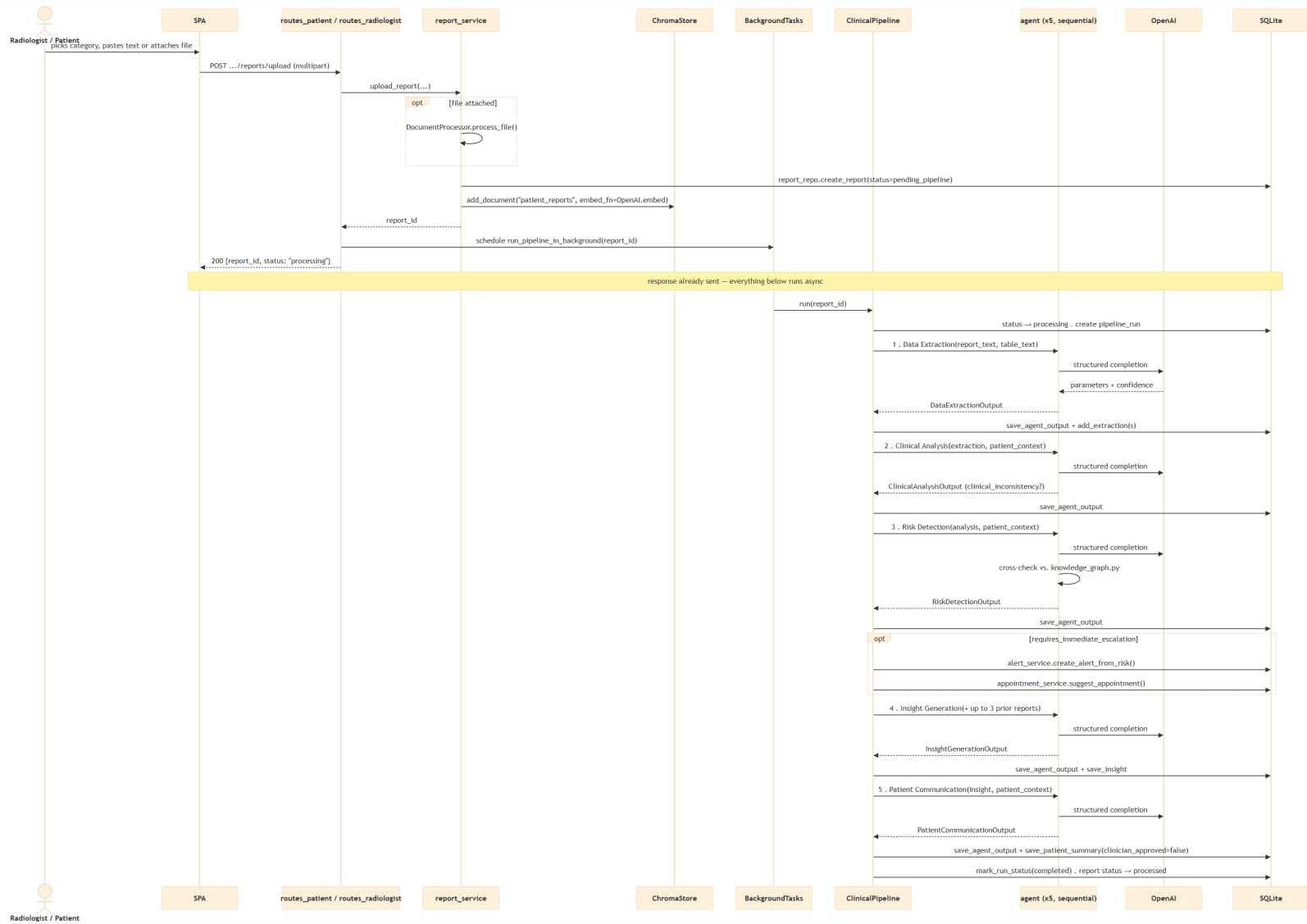
4.1 Login

Session-cookie auth — pbkdf2 password check, no JWTs, no refresh tokens. A patient's session additionally carries their `patient_id` so every later request is scoped without a lookup.



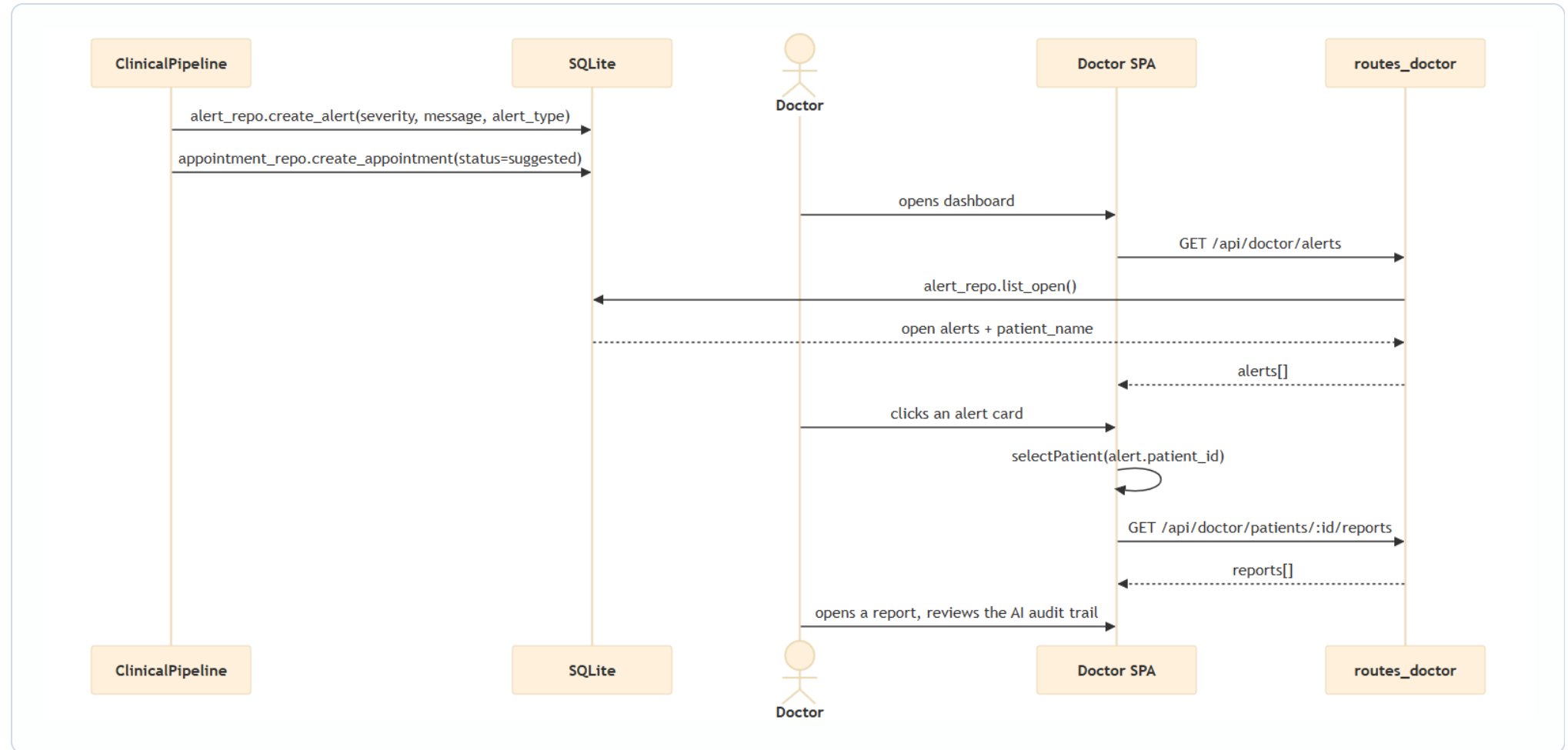
4.2 Report upload & the 5-agent pipeline

The upload request returns as soon as the report row exists and the file is embedded — the pipeline itself runs after the response is already sent, via FastAPI's BackgroundTasks, on its own DB connection.



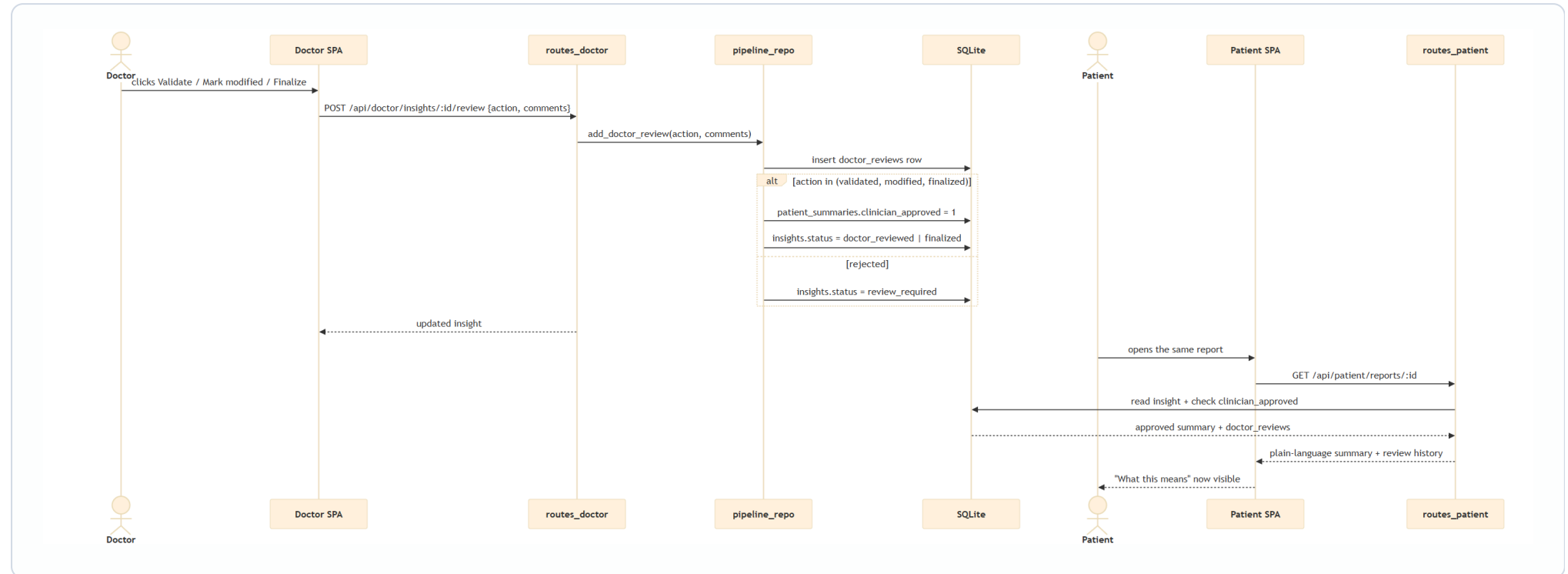
4.3 Critical finding → doctor escalation

`appointment_service.suggest_appointment` never sets a `doctor_user_id` — the escalation is patient-linked, not doctor-linked, so what actually routes a doctor to the patient is the Alerts panel, not the appointments list.



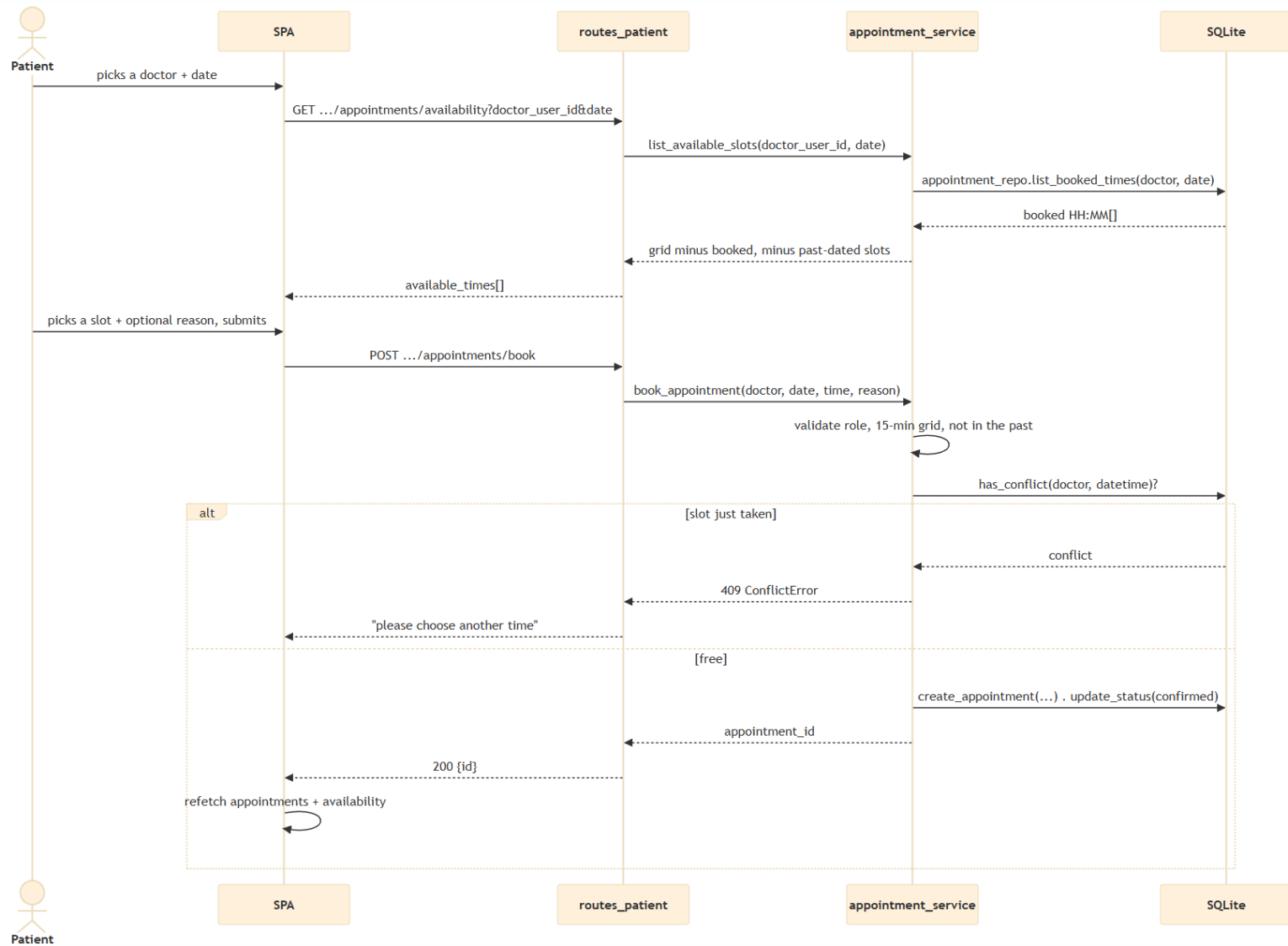
4.4 Doctor review → patient summary reveal

`patient_summaries.clinician_approved` starts false regardless of what the Patient Communication agent claims about its own output — this row flip is the only thing standing between a draft summary and a patient seeing it.



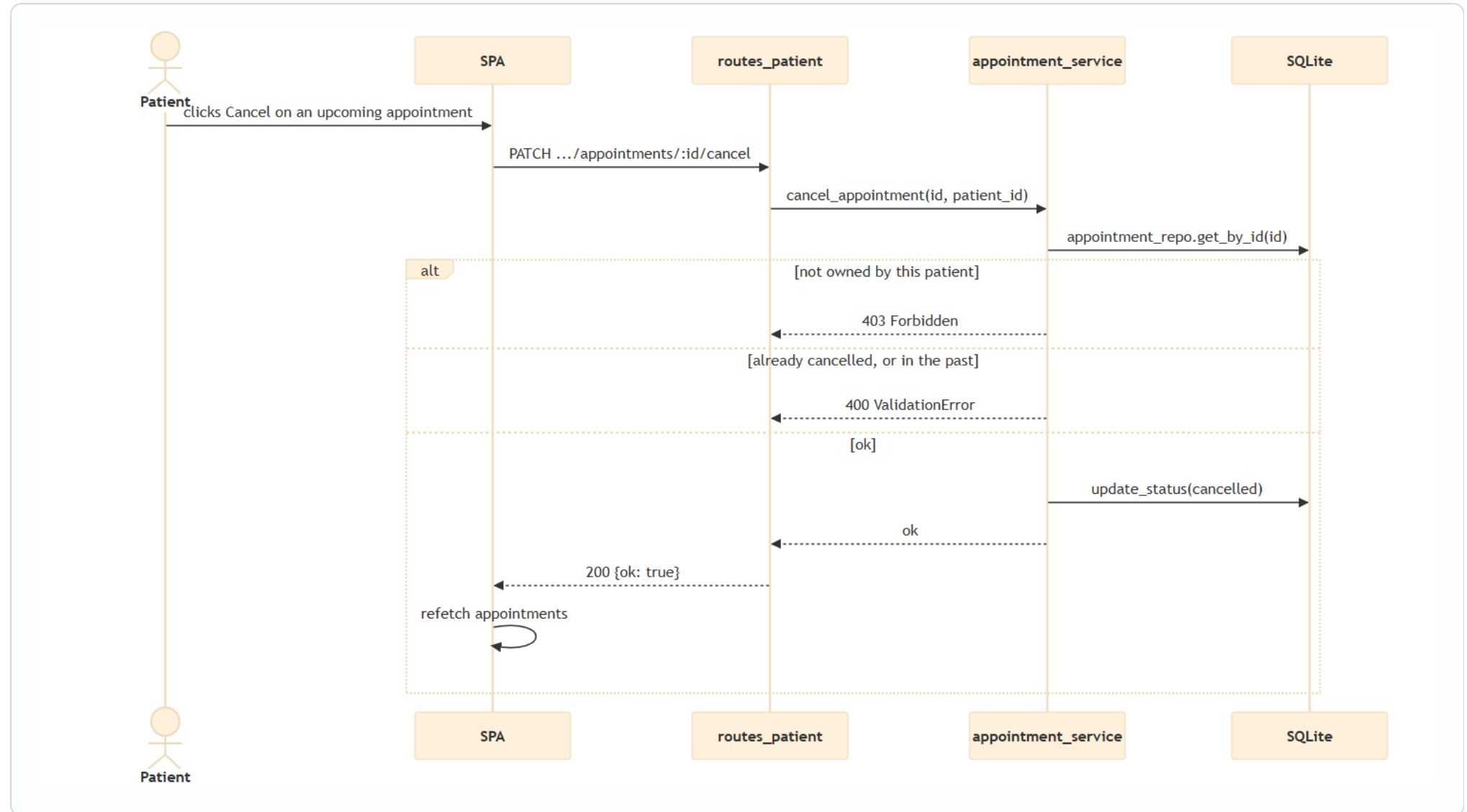
4.5 **Patient books an appointment**

Availability is computed, not stored: the fixed 10:00 AM–1:00 PM / 4:00–7:00 PM 15-minute grid minus whatever's already booked. The same conflict check runs again on submit, closing the gap between two patients viewing the same open slot at once.



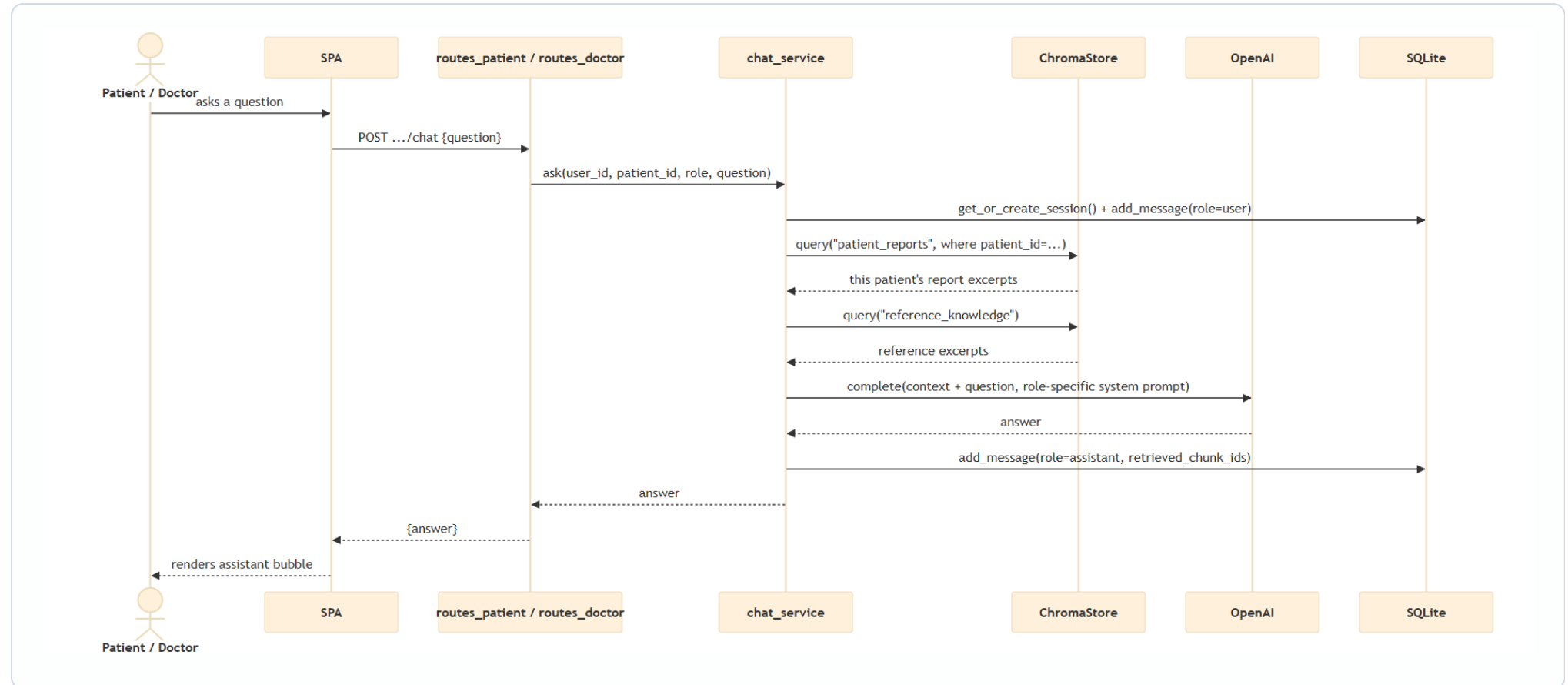
4.6 Patient cancels an appointment

Ownership, status, and tense are all checked server-side — a patient can't cancel someone else's slot, cancel twice, or "cancel" something that already happened.



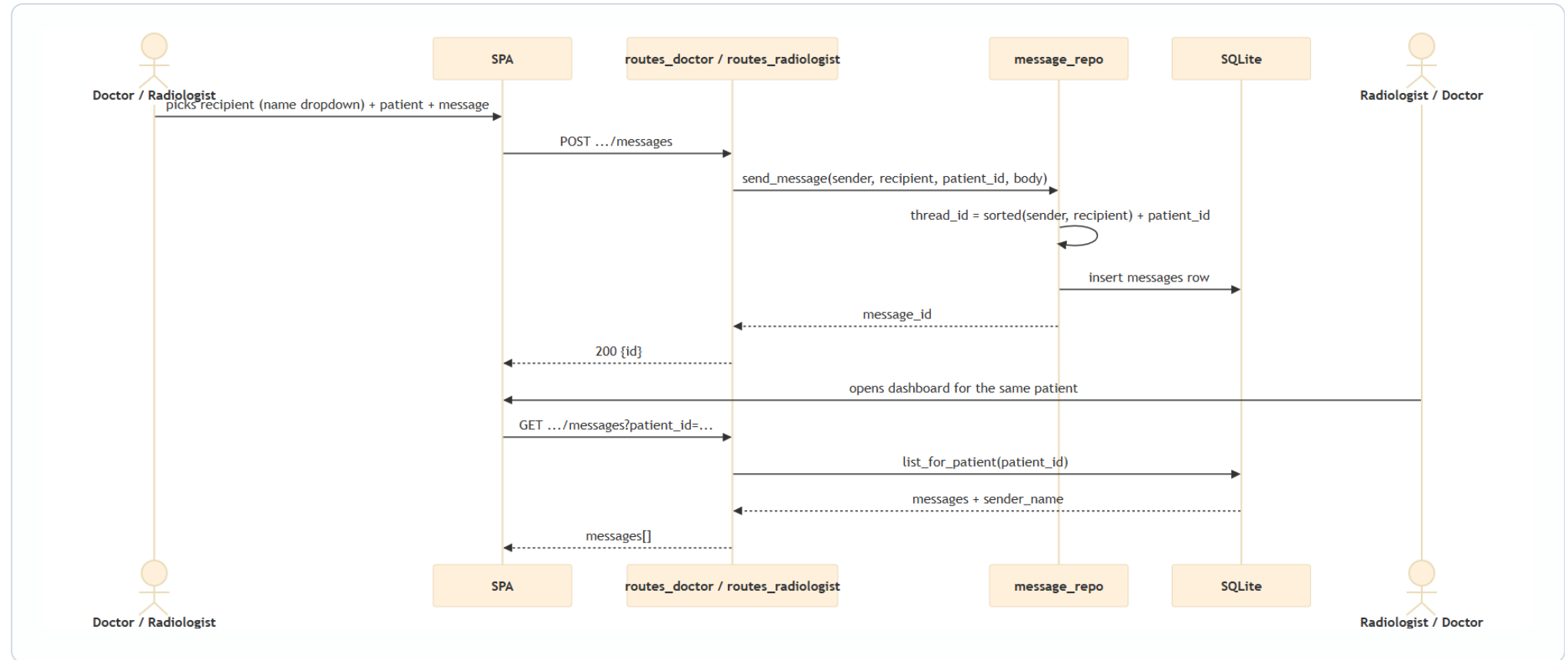
4.7 AI assistant chat (RAG)

The same endpoint shape backs both "AI Health Assistant" and "AI Clinical Assistant" — what changes is the system prompt and that a patient's `patient_id` comes from their session while a doctor's comes from whichever patient is currently selected.



4.8 Doctor ↔ radiologist messaging

Messages are grouped into threads by a deterministic key — the two user IDs sorted, plus the patient ID — so either side listing "messages about this patient" hits the same rows without a separate thread table.



Generated from the current app/ codebase — backend/ + frontend/.

No real EHR/EMR or calendar system is ever contacted — appointments and messaging are simulated in-app by design.

05 Deployment

Sections 01–04 cover what the system is made of and how a request moves through it. This system also runs live — one EC2 instance, one Docker container, one persistent EBS volume, provisioned by Terraform and deployed by GitHub Actions, with secrets in SSM Parameter Store and no SSH key on the box.

 **LIVE** — <http://ec2-63-185-251-88.eu-central-1.compute.amazonaws.com>

The full deployment write-up — rationale, first-time setup, CI/CD, operations, and this specific deployment's details — is its own document: [docs/deployment-architecture.pdf](#).

05 Deployment diagram

